

Common Agile Practices in Software Processes

José Fortuna Abrantes, Guilherme Horta Travassos
COPPE/Systems Engineering and Computer Science Program
Federal University of Rio de Janeiro
Rio de Janeiro, Brazil
{jfa, ght}@cos.ufrj.br

Abstract— Objective: to investigate studies about software processes looking for practices which can be used to obtain agility in software processes. Method: A systematic review including seven search engines was executed in Feb/2010. To apply the defined criteria to select papers and extract information regarding working practices bringing agility to software processes. Results: from 6696 retrieved papers, 441 were selected to support the identification of 236 occurrences of 51 distinct practices associated with the concept of agility. Their descriptions were deeply analyzed and consolidated. After discarding those which appeared in the technical literature in a small amount of papers, 17 agile practices were identified. Conclusion: although further studies are necessary to evaluate the efficacy of these 17 agile practices, 12 of them have been more commonly approached in the software projects and could be primarily considered: test driven development, continuous integration, pair programming, planning game, on-site customer, collective code ownership, small releases, metaphor, refactoring, sustainable pace, simple design and coding standards.

Keywords- agile practices, agile software processes, agile methods, systematic review, evidence based software engineering.

I. INTRODUCTION

Nowadays, software development practices adopted by organizations are strongly challenged by the evolving business environments. In a volatile business context, software organizations come across a new reality and have to deal with uncertainties and quick changes in business environments, by re-engineering their methods. Many sources of change are frequently unpredictable. Slowness or delays to accommodate uncertainty and rapid changes inevitably leads to the obsolescence of software and customer dissatisfaction [1]. It has been demonstrated that values, principles and agile practices bring benefits to many organizations [2]. Values bring purpose to practices. Practices are evidence of values, which are universal and expressed at a higher level. Practices are clearer and more specific. Bridging the gap between values and practices are principles, which are domain-specific guidelines. Just as values bring purpose to practices, practices bring accountability to values [3]. Practices are activities that implement the principles governing the processes or methods. These in turn are ideas, understandings or goals that are behind the practices [4].

In the last 16 years, an interesting changing in the software development processes was the introduction of the “agile” perspective. However, software development teams

need support to choose the right combination of agile practices based on their needs [5]. The aim of this study is to investigate software practices recommended into the context of agile approaches for software development. A research protocol has been formalized to conduct a systematic literature review. The searches were conducted in February 2010. The data were analyzed and the initial results summarized in this short paper.

II. RESEARCH METHOD

The objective of this secondary study is to identify the software practices usually used into the context of agile approaches for the development of software. The formulated question was: what are the software practices that can be considered agile into the context of approaches to develop software? The problem is finding and identifying agile practices for software development. The application of the results is intended to serve as a basis to support researches involving software practices that can be used as alternatives to embed agility into software processes. The population is the set of software development projects. The intervention is the agile software development processes. There is no comparison. The outcome is a set of agile practices. Three papers were used as control: [6, 7, 8]. The sources are collected from the following digital databases, including the conferences, journals and technical reports indexed by Compendex EI, IeeeXplore, Inspec, Web of Science, Scopus, Science Direct, and ACM digital library.

The keywords for population are "software projects", "software systems", "software development", "software engineering". The keywords for intervention are "agile approaches", "agile processes", "agile methods", "agile methodologies", "agile development", "agile software development", "agile projects". The keywords for outcome are "practices", "software practices", "agile practices", "sub practices", "agile techniques", "techniques".

The inclusion and exclusion criteria are: the documents should be available on the web; the documents should include agile software practices into the context of agile approaches for software development; the documents should provide a description of the agile practices they behold. In order to select studies, a researcher will apply the search strategy for identification of potential documents. The documents identified by the researcher will be selected through reading and verification of the inclusion and exclusion criteria. Subsequently, the documents excluded from the list will be evaluated by a second researcher. In case of conflict, the document will be included. Finally, the

documents will be read by researchers to extract information about software practices into the context of agile software development approaches. It is assumed the sources of the documents are reliable, and the texts have undergone external reviews that serve as a filter so that they present enough quality to contribute to this secondary study.

The following information will be extracted from each paper selected after running the selection process: document title, author (s), source, year of publication and name and description of the agile software practices. The results will be tabulated. Analysis will be carried out to identify similarities and most relevant or most significant practices for software processes into the context of agile software development. The frequency in which each practice has been reported by different authors will be considered. If authors write more than one paper, repeated counts will be avoided. The search string taken as the basis for all search engines, structured according to Pai et al. [9] was: ("software projects" or "software systems" or "software development" or "software engineering") AND ("agile approaches" or "agile processes" or "agile methods" or "agile methodologies" or "agile development" or "agile software development" or "agile projects") AND ("practices" or "software practices" or "agile practices" or "subpractices" or "agile techniques" or "techniques").

III. RESULTS AND DISCUSSION

The searches execution with the aforementioned engines returned 6696 references, published between 1998 and 2010. Repetitions were eliminated, keeping the remaining paper counted for the digital library with the largest number of items recovered. All the controls were retrieved. From 5093 references without repetitions, titles and summaries were assessed for each retrieved reference, and references that clearly did not address issues concerned with the research were excluded. After the removal of such items, 441 references were selected for the study.

Following the criteria established in the research protocol, 236 occurrences of practices were extracted from 24 papers published between 2001 and 2009 [6, 7, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31] and tabulated accordingly. In order to avoid bias, technical papers concerning with performance studies or specific issues of a particular practice were not considered. From these, some practices are associated to one or more commercial agile methods while others are not associated with any method. Other papers approached agile practices, but did not present an adequate description of its meaning. In order to identify repetitions, names and descriptions of the 236 occurrences of agile practices collected were analyzed and grouped, leading to 51 different agile practices. One can observe significant increase of technical papers published in 2009 meeting the research protocol criteria, as well as the growing interest by the scientific community on agile approaches, considering the results of other secondary study regarding agility characteristics previously undertaken by the authors of this study.

From the 51 different agile practices identified, 17 of them were approached in the technical papers more than once and they were selected to be further analyzed. The descriptions for each practice from different papers were consolidated as follows: *Whole Team*- Refers to the practice of including all necessary skills and perspectives on the team for it to succeed, emphasizing the sense of team and all team members sharing the purpose and supporting each other. Customers, end-users and other business stakeholders should have direct involvement on the project, in order to understand the system's behavior early in the lifecycle.

Project Visibility- Agile projects strive to provide immediate status and feedback to the teams. One can create project dashboards on the web that hosts, at any point in time, all status and measurements regarding the progress of the project. Advancement of the project in relation to the user stories that the teams have committed to deliver at the end of the iterations should be included. Models should be made accessible for the entire teams.

Continuous integration- The members of a team should frequently integrate their work, every time new changes or a task is completed, to reveal integration problems and to detect system failures as soon as possible. Usually each person integrates at least daily. All tests must still pass after integration or the new code should be discarded.

Test Driven Development- Every programmer writes test cases before they write their production code. Developers write unit tests before coding and encourage customers to write acceptance test cases. Upon initial execution of such a test case, it will fail as there is no corresponding code that implements the particular feature or condition being tested. Then the developer compose just enough code for the test to pass or to satisfy the current goal, followed by refactoring in order to improve readability and remove duplications.

Refactoring- This is a practice for restructuring an existing body of code, or constantly improving its understandability and maintainability, as well as its design, altering its internal structure without changing its external behavior nor the system functionality. The different forms of refactoring involve simplifying complex statements, abstracting common solutions into reusable code, and the removal of duplicate code. When code is refactored, it must still pass all the unit test cases in the code base.

Pair programming- All source code produced must be written by two people working at the same time in the same machine. The programmers alternate the roles of driver and navigator throughout the day. The driver has control of the keyboard and mouse; s/he implements the code, explaining it to the navigator. The driver's major responsibilities are development tests and algorithm construction, whereas the obligations of the navigator are to watch for both syntax and semantic defects, deciding if this is the best way to implement the functionality.

Planning Game- Together, the developers and the customers play the "Planning Game" in which the customer chooses those User Stories that comprise the most important content for a short, incremental deliverable. Each short implementation increment is accepted and tried by the customer. Then, the remaining User Stories are re-examined

for possible requirement and/or priority changes and the Planning Game is re-played for the next implementation increment. Planning is continuous and progressive.

Small releases- Release working software frequently. This practice shortens release cycle to speed the feedback from the customer. Given that requirements often change, one keeps release cycles short and ensures that each release produces a useful software system that generates business value for the customer. An initial version of the system is put into production quickly, after little iteration. At the end of each release, the customer reviews the interim product; identify defects and adjusting future requirements.

Metaphor- This practice presents a simple shared story which explains the essence of how the system works to give both developers and customers a common understanding of the project. In a sense, the metaphor serves as the high-level software architecture. While thinking about an appropriate metaphor, developers must expand their perspective and analysis of the developed application.

Simple design- The emphasis of this practice is on designing the simplest possible solution that is implementable at that moment. Unnecessary complexity and extra code shall be immediately removed. One should not add additional features to your artifacts unless they are justifiable. Programmers should not try to predict future needs. The most cost-effective development approach should focus on solving today's problems rather than designing for future changes.

Collective Code Ownership- The code repository should be freely accessible to all programmers, who are encouraged to make changes in all the code, anywhere and at anytime they feel necessary, without asking 'permission' from anyone. Any programmers' pair whom sees an opportunity to add value to any portion of the code can do so any time. As the code is accessible to more minds, programmers must examine code that is written by others.

On-site customer- This practice indicates that customer should be a member of the development team. To clarify and validate the requirements and set priorities, an on-site customer representative works with the team. Thus a customer works close together with the developers all the time, to answer the questions, resolve disputes, set small scale priorities, perform acceptance tests, and ensure that development is progressing as expected.

Sustainable Pace- This practice emphasizes working "only as many hours as you can be productive and only as many as you can sustain". Work no more than 40 hours a week as a rule, no more than eight hours every day, without working overtime a second week in a row. During crunch periods when overtime is worked, the artifacts that are produced are poor in quality. Requirements should be selected for each iteration, such that developers do not need to put in overtime.

Open workspace- Developers work in a common workspace; a large room with small cubicles is preferred. Pair programmers should be placed in the center of the space. Workplace layouts should have common areas that facilitate open communication. An alternative set with individual workstations around the periphery and common development machines in the center could be appropriate.

Coding standards- Because developers code different system parts with various team members, coding standards are a must. A coding standard makes the code easier to understand, increase readability and improves consistency among team members. The standard should be easy to follow and to be voluntarily adopted. It is agreed upon by the team to ensure that communication is made through code and let developers to easily understand each other's code.

Product Backlog- This practice includes the tasks for creating the Product Backlog list, and controlling it consistently during the process by adding, removing, specifying, updating, and prioritizing items. Product Backlog defines everything that is needed in the final product based on current knowledge. It comprises a prioritized and constantly updated list of business and technical requirements for the system being built or enhanced.

Stand-up meetings- they are quick meetings (approximately 15 minutes) organized to keep track of the project progress, to highlight issues and to organize the daily activities. Each team member briefly states what he or she has been working on and what progress has been made. The standing component of the meeting tended to ensure that everyone be alert and attentive, besides intending not to go beyond the 15 minutes, encouraging the participants to conclude it quickly.

From these, 12 practices have been addressed by technical papers published in at least 6 different years in the time interval of the selected publications: *test driven development, continuous integration, pair programming, planning game, on-site customer, collective code ownership, small releases, metaphor, refactoring, sustainable pace, simple design and coding standards*. Other 5 practices, completing 17 which were approached in the papers more than once were: *whole team, project visibility, stand-up meetings, open workspace and product backlog*.

IV. THREATS TO VALIDITY

The threats to validity of this study are associated with the criteria that governed the construction of the search string. The goal was to avoid the loss of relevant documents. Therefore, due to the choice of terms to form the search strings, there is a risk that relevant studies have been omitted. This study identified a set of agile practices based on findings in the technical literature. One can not be sure that the same set of practices would be found if the identification were based on findings in real agile software projects, which possibly could lead to different results. So, may be there are other agile practices that could be considered if a more industrial view is used in this study.

V. CONCLUSIONS AND FUTURE WORKS

The performance of all practices, in terms of agility, depends on the environment in which it is applied, the project context, the way chosen to implement it in the process and the intensity with which it is applied and upheld by the team. Among the 51 distinct practices identified in this secondary study, several of them were targets of interest to researchers only occasionally, the vast majority before 2004. The proposal now is to continue this research work, considering the aforementioned 17 agile practices. The

intention is to run a primary study to evaluate them. The primary study will be structured to capture and consider any additional practices that any of its participants want to add, since they present descriptions of the meaning of the practices they suggest.

ACKNOWLEDGMENTS

This work is into the context of *eSEE* (experimental Software Engineering Environment) research project supported by CNPq – Brazilian Research Council, FAPERJ and COPPETEC Foundation.

REFERENCES

- [1] Ktata, Oualid. Lévesque, Ghislain.: Agile development: Issues and avenues requiring a substantial enhancement of the business perspective in large projects, C3S2E-09, May 19-21, Montreal [QC, CANADA] Editor: B C. DESAI (2009).
- [2] Glazer, Hillel.: Love and Marriage: CMMI and Agile Need Each Other, CrossTalk: The Journal of Defense Software Engineering. Volume 23, No. 1, Jan/Feb. (2010).
- [3] Beck, K. Andres, Cynthia.: Extreme Programming Explained: Embrace Change, Second Edition, Addison Wesley Professional (2004).
- [4] Jiang, Li. Eberlein, Armin.: An Analysis of the History of Classical Software Development and Agile Development, In: IEEE International Conference on Systems, Man, and Cybernetics San Antonio, TX, USA – October (2009).
- [5] Abbas, Noura. Gravell, Andrew M. Wills, Gary B.: Using Factor Analysis to Generate Clusters of Agile Practices, In: AGILE Conference, August 9 - 13, Orlando, Florida (2010).
- [6] Abrahamsson, P. Salo, O. Ronkainen, J. Warsta, J.: Agile Software Development Methods. Review and Analysis, Espoo. VTT Publications 478 (2002).
- [7] Cohen, D. Lindvall, M. Costa, P.: An Introduction to Agile Methods, in: M.V. Zelkowitz (Ed.), Advances in Computers, Advances in Software Engineering, vol. 62, Elsevier, Amsterdam (2004).
- [8] Meso, Peter. Jain, Radhika.: Contemporary practices in systems development. Agile Software Development: adaptive systems principles and best practices, Information Systems Management, summer (2006).
- [9] Pai, M. McCulloch, M. Gorman, J.D. et al. : Systematic Reviews and meta-analyses: An illustrated, step-by-step guide, The National Medical Journal of India, vol. 17, n.2 (2004).
- [10] Koehnemann, Harry. Coats, Mark. (2009), "Experiences Applying Agile Practices to Large Systems", In: 2009 Agile Conference.
- [11] Fruhling, Ann. De Vreede, Gert-Jan. (2006) "Field Experiences with eXtreme Programming: Developing an Emergency Response System", Journal of Management Information Systems / Spring Vol. 22, No. 4. pp. 39-68.
- [12] Paige, Richard F. Brooke, Phillip J. (2005) "Agile Formal Method Engineering", J. Romijn, G. Smith, and J. van de Pol (Eds.): IFM 2005, LNCS 3771, pp. 109–128.
- [13] Jureczko, Marian. (2008) "The Level of Agility in Testing Process in a Large Scale Financial Software Project", In: Software engineering techniques in progress, Oficyna Wydawnicza Politechniki Wroc'awskiej, 139-152.
- [14] Xu, Bin. (2009) "Towards High Quality Software Development with Extreme Programming Methodology: Practices from Real Software Projects", College. of Computer Science & Information Engineering, Zhejiang Gongshang University, Hangzhou China.
- [15] Stolberg, Sean. (2009) "Enabling Agile Testing Through Continuous Integration", 2009 Agile Conference.
- [16] 9- Martin, Angela. Biddle, Robert. Noble, James. (2009) "XP Customer Practices: A Grounded Theory", 2009 Agile Conference.
- [17] 10- Zhou, Yinghua. (2009) "UniX Process, Merging Unified Process and Extreme Programming to Benefit Software Development Practice", 2009 First International Workshop on Education Technology and Computer Science.
- [18] Cannizzo, Fabrizio. Marcionetti, Gabriela. Moser, Paul. (2008) "The Toolbox Of A Successful Software Craftsman", 15th Annual IEEE International Conference and Workshop on the Engineering of Computer Based Systems.
- [19] Huo, Ming. Verner, June. Zhu, Liming. Ali Babar, Muhammad. (2004) "Software Quality and Agile Methods", Proceedings of the 28th Annual International Computer Software and Applications Conference (COMPSAC'04).
- [20] Maurer, Frank. Martel, Sebastien. (2002) "Extreme Programming Rapid Development for Web-Based Applications", IEEE Internet Computing, January - February.
- [21] Paulk, Mark C. (2001) "Extreme Programming from a CMM Perspective", IEEE Software, November/December.
- [22] Vejjandla, Pavan K. Sherrell, Linda B. (2009) "Why an AI Research Team Adopted XP Practices", Proceedings of the 47th Annual Southeast Regional Conference, ACM-SE 47, March 19-21, Clemson, SC, USA.
- [23] Concas, Giulio. Francesco, Marco Di. Marchesi, Michele. Quaresima, Roberta. Pinna, Sandro. (2008) "Study of the Evolution of an Agile Project Featuring a Web Application Using Software Metrics", A. Jedlitschka and O. Salo (Eds.): PROFES 2008, LNCS 5089, pp. 386-399
- [24] Hazzan, Orit. Tomayko, Jim. (2003) "The Reflective Practitioner Perspective in eXtreme Programming", Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics), 2753, 51-61.
- [25] Xiaohua?Wang. Zhi, Wu. Ming, Zhao. (2008) "The Relationship between Developers and Customers in Agile Methodology", 2008 IEEE International Conference on Global Software Engineering.
- [26] Aiello, G. Alessi, M. Bruccoleri, M. D'Onofrio C. Vella, G. (2007) "An Agile methodology for Manufacturing Control Systems development", Engisud S.p.a., Palermo, Italy.
- [27] Svensson, Harald. H"ost, Martin. (2005) "Introducing an Agile Process in a Software Maintenance and Evolution Organization", Proceedings of the Ninth European Conference on Software Maintenance and Reengineering (CSMR'05).
- [28] Mills, David. Sherrell, Linda. Boydston, Jeff. Wei, Guoqing. (2005) "Experiences Using Agile Software Development for a Marketing Simulation", Dept. of Comput. Sci., Memphis Univ., TN USA.
- [29] McKinney, Dawn. Denton, Leo F. (2005) "Affective Assessment of Team Skills in Agile CS1 Labs: The Good, the Bad, and the Ugly", SIGCSE '05, February 23-27, 2005, St. Louis, Missouri, USA.
- [30] Ramachandran, Vinay. Shukla, Anuja. (2002) "Circle of Life, Spiral of Death: Are XP Teams Following the Essential Practices?", Dept. of Comput. Sci., North Carolina State Univ., Raleigh NC USA
- [31] Williams, Laurie. Upchurch, Richard. (2001) "Extreme Programming for Software Engineering Education?", 31st ASEE/IEEE Frontiers in Education Conference, October 10 - 13, 2001 Reno, NV.